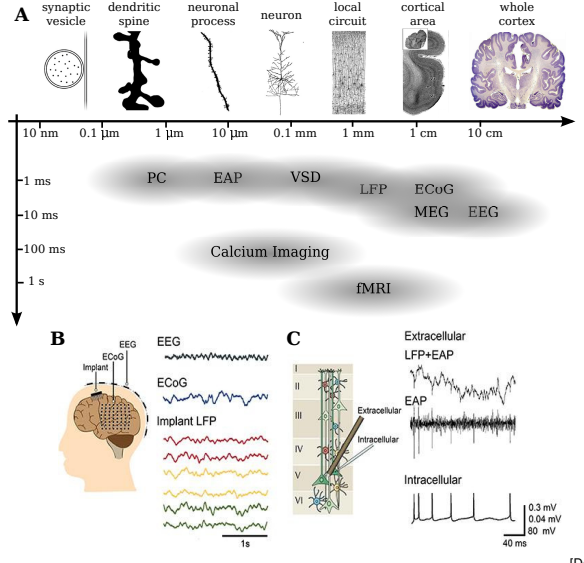


INTRODUCTION TO NEST SIMULATOR

Large-scale simulation of spiking neural networks

October 14th, 2026 | Charl Linssen | Jülich Supercomputing Centre (JSC), Germany

MULTI-SCALE BRAIN STRUCTURE AND DYNAMICS



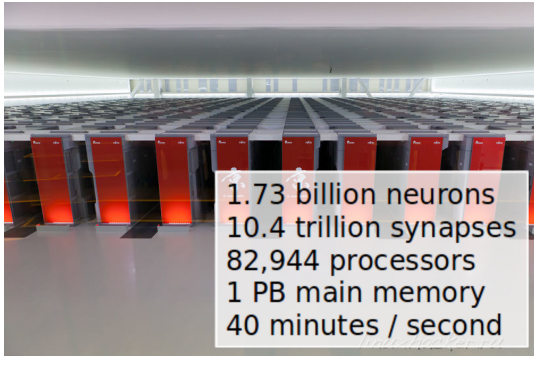
[Dahmen, 2017]

NEST SIMULATOR DESIGN GOALS

- Main focus: spiking point neurons and biologically realistic synaptic plasticity.
- Models can be very simple or contain a lot of biological detail.
- NEST development is always driven by scientific needs.
- High accuracy and flexibility
  - Spike interaction in continuous time available for suitable neuron models
  - Extremely scalable: same code from laptop to supercomputers
- Constant quality assurance
  - Automated unit test suite included in NEST build
  - Continuous integration for new code
  - Peer review for all code contributions

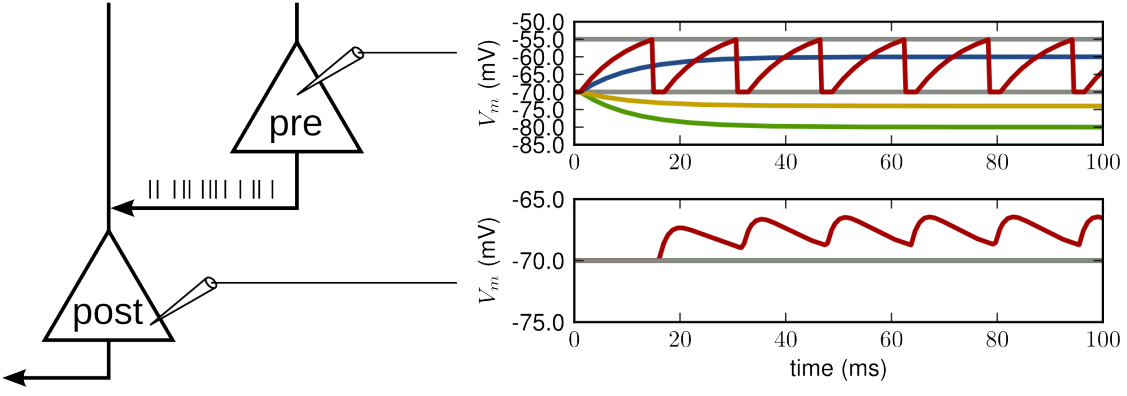
NEST SIMULATOR SOFTWARE ARCHITECTURE

- Highly efficient C++ core with a Python frontend
- Hybrid parallelization (OpenMP+MPI)
- Exact integration is used where possible
- Same code from laptops to supercomputers



NEURONAL SIMULATIONS IN NEST

A simulation in NEST mimics a neuroscientific experiment.

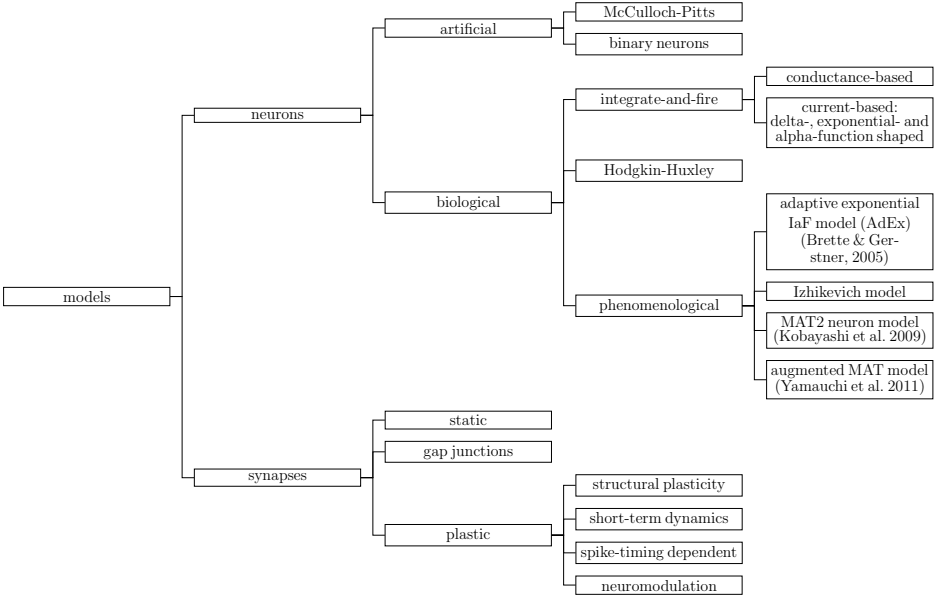


Except that you can easily measure and control every neuron and connection in the network!

THREE MAIN COMPONENTS OF A NEST SIMULATION

- **Nodes**
  - Neurons – Devices (– Sub-networks)
  - Have dynamic state variable(s) that changes over time ($V_\mathrm{m}(t)$)
  - Can be affected by events (spikes)
- **Events**
  - Pieces of information of a particular type (e.g., spike, voltage or current event)
  - Recording devices: `'spike_recorder'`, `'voltmeter'`, `'multimeter'`
- **Connections**
  - Communication channels for the exchange of events
  - Directed (from source node to target node)
  - Weighted (how strongly does an event influence the target node)
  - Delayed (length of transmission duration between source and target)
  - Connections are created using one global `Connect` function

NEST NEURON AND SYNAPSE MODELS



NEURON MODELS

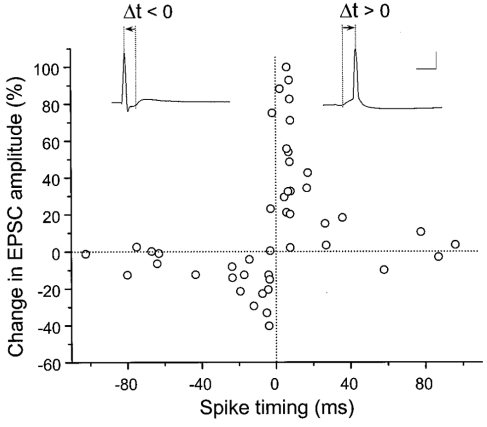
- Integrate-and-fire models (`iaf_`)
  - Current-based (`iaf_psc`)
  - Conductance-based (`iaf_cond`)
  - Different post-synaptic shapes (`_alpha`, `_exp`, `_delta`)
- Single compartment Hodgin-Huxley models (`hh_`)
- Adaptive exponential integrate-and-fire models (`aeif_`)
- Allen Institute generalised leaky integrate-and fire family (`gif_`)
- Hill-Tononi model (Hill & Tononi 2005)
- Neuron models with >1 compartments (“mesocompartmental”)
- . . . and many more!

SYNAPTIC PLASTICITY MODELS IN NEST

- Biological substrate of learning and memory
- ”Synaptic update rules” or ”learning rules” describe the change of connection strength w_{ij} for a synapse from neuron j to neuron i
- Different time scales for inducing a change and the persistence of the change, e.g.,
  - Short-term plasticity: induced in less than a second, lasts less than a second
  - Long-term depression (LTD) and long-term potentiation (LTP): induced in a few seconds, can last for more than an hour
- Dependence on, e.g., pre- and postsynaptic spike times, or additionally a third factor as modulatory signal, for example, neuromodulation, eligibility traces, voltage

Morrison, Diesmann & Gerstner (2008)

SYNAPTIC PLASTICITY MODELS IN NEST



Bi & Poo (1998)

- All-to-all and several nearest-neighbour variants
- Triplet STDP (Pfister & Gerstner, 2006)
- Voltage-based STDP (Clopath et al., 2010)
- Dopamine-modulated STDP (Potjans et al., 2010)
- Short term depression and facilitation (Tsodyks, Uziel & Markram, 2000)
- Voltage-based learning rules (Clopath et al., 2020; Urbanczik & Senn, 2014)
- Structural plasticity (Butz & van Ooyen, 2013)
- E-prop (biologically plausible approximation of back-propagation through time) (Bellec et al., 2020)

STIMULATION DEVICES

Spike generators:

- `spike_generator` spikes at prescibed points in time
- `poisson_generator` spikes according to a Poisson distribution
- `gamma_sup_generator` spikes according to a Gamma distribution

Current generators

- `ac_generator` provides a sine-shaped current
- `dc_generator` provices a constant current
- `step_current_generator` provides a step-wise constant current
- `noise_generator` provides a random noise current

RECORDING DEVICES

- `spike_detector` records incoming spikes
- `multimeter` records analog quantities (potentials, conductances, ...)
- `voltmeter` records the membrane potential
- `correlation_detector` records pairwise cross-correlations between the spiking activity of neurons
- `weight_recorder` records the weight of connections

can record to memory, file, or stream data!

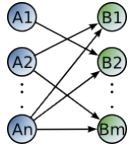
SPECIFICATION OF CONNECTIVITY

- One-to-one
- All-to-all
- Fixed in-, out-degree
- Fixed total number (*N*)
- Pairwise Bernoulli (*p*)

```
A = Create('iaf_psc_alpha', n)
B = Create('spike_recorder', n)
Connect(A, B, 'one_to_one')
```




```
A = Create("iaf_psc_alpha", n)
B = Create("iaf_psc_alpha", m)
Connect(A, B, {'rule': 'fixed_indegree',
               'indegree': N})
```



RANDOMIZATION OF SYNAPSE PROPERTIES

Dozens of standard distributions are natively supported.

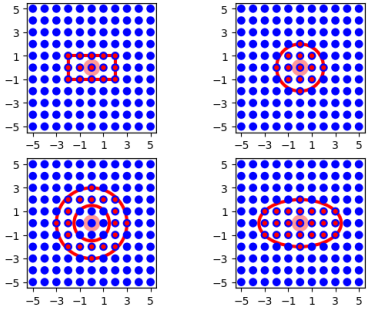
```
delay_dist = {'distribution': 'uniform',
              'low': 0.8, 'high': 2.5}

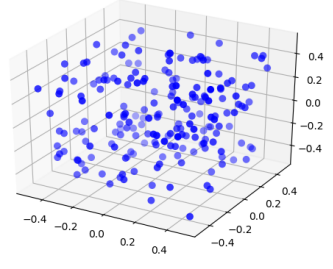
alpha_dist = {'distribution': 'normal_clipped',
              'low': 0.5, 'mu': 5.0,
              'sigma': 1.0}

syn_dict = {'model': 'stdp_synapse',
            'weight': 2.5,
            'delay': delay_dist,
            'alpha': alpha_dist}
```

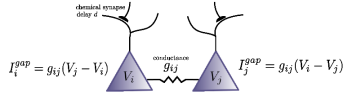
TOPOLOGY

- Lay out elements on grids or at arbitrary points in space (2D or 3D)
- Elements can be neurons or combinations of neurons and devices
- Connect neurons in a position- and distance-dependent manner
- Set periodic boundary conditions
- Choose whether to allow self-connections (autapses) or multiple connections (multapses)
- Distance-dependent or random weights and delays





GAP JUNCTIONS

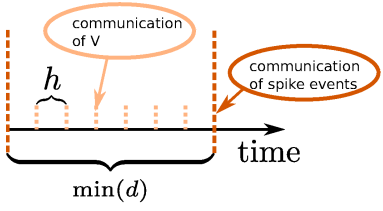


Neuron i (hh_psc_alpha_gap)

$$y_i'(t) = f_i(y_i(t)) \ , \ y_i(t_0) \text{ given}$$

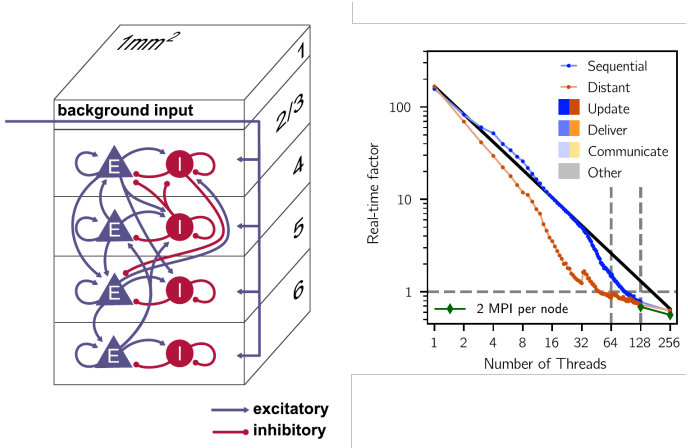
$$\frac{V_i'}{C_m} = -I_i^{ionic}(V_i, m_i, h_i, n_i, p_i) + I_i^{applied}(I_i^{ex}, I_i^{in}) + I_i^{gap}(V_i, V_j)$$

- at each time point neuron i needs membrane potential of neuron j
- large system of differential equations
- naïve: communication of V in each step
- better: Jacobi waveform relaxation



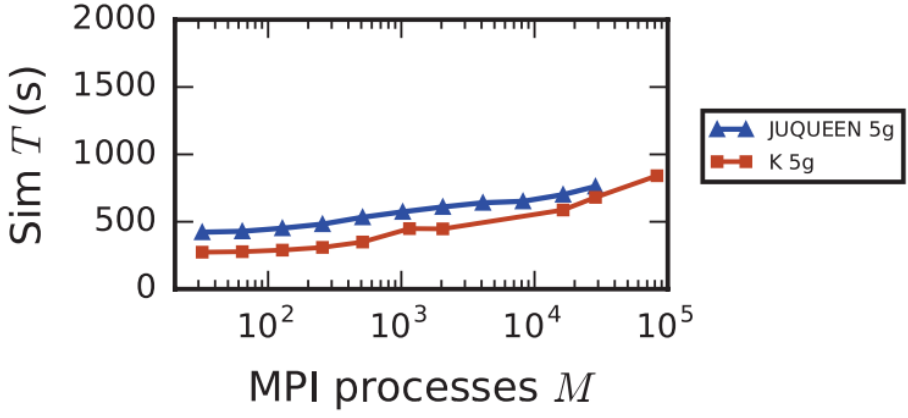
Hahne et al. (2015). A unified framework for spiking and gap-junction interactions in distributed neural network simulations. Frontiers in Neuroinformatics. 9:22

PERFORMANCE: STRONG SCALING



Potjans-Diesmann Microcircuit ($\sim 80,000$ neurons) on AMD EPYC Rome
Kurth, Senk, Terhorst, Finnerty & Diesmann (2022) Neuromorph. Comput. Eng. [Figure modified]

PERFORMANCE: WEAK SCALING



Jordan, Ippen, Helias, Kitayama, Sato, Igarashi, Diesmann & Kunkel (2018) Front. Neuroinform.

HOW TO USE NEST?

- The Python interface PyNEST

```
import nest
n = nest.Create('iaf_psc_exp', 8000, {'V_m': -65.})
sd = nest.Create('spike_recorder')
nest.Connect(n, sd)
...
nest.Simulate(1000.)
```

- The multi-simulator Python interface PyNN

```
from pyNN import nest
nest.setup()
neuron_t = nest.native_cell_type('iaf_psc_exp')
pop_exc = nest.Population(8000, neuron_t, {})
pop_exc.initialize(V_m=-65.)
...
nest.run(1000.)
```

A FULL EXAMPLE

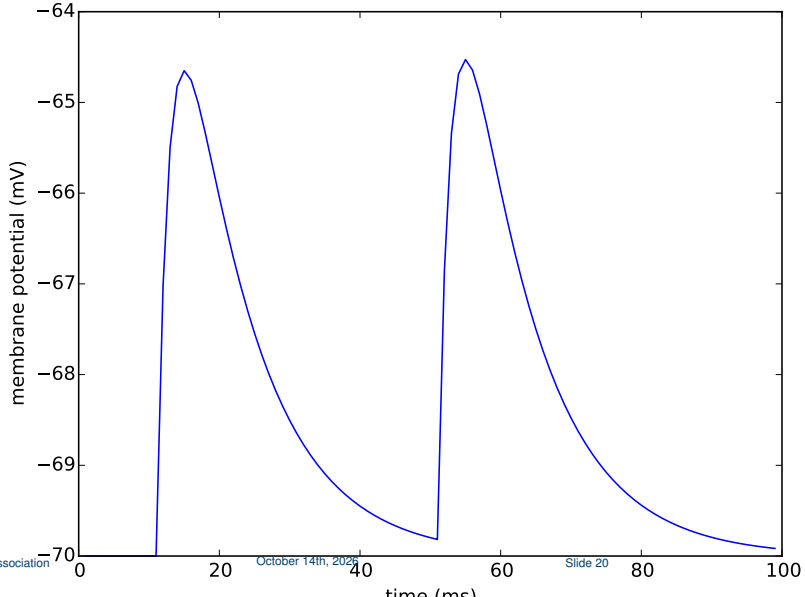
```
import nest                                      # import NEST module
neuron = nest.Create('iaf_psc_exp')              # create a neuron
voltmeter = nest.Create('voltmeter')             # create a voltmeter
spikegenerator = nest.Create('spike_generator') # create a spike generator
nest.SetStatus(spikegenerator, {'spike_times': [10., 50.]}) # let it spike

# connect spike generator and voltmeter to the neuron
nest.Connect(spikegenerator, neuron, syn_spec={'weight' : 1E3})
nest.Connect(voltmeter, neuron)

nest.Simulate(100.) # run the simulation

# read out recording time and voltage from voltmeter and plot them
times = nest.GetStatus(voltmeter)[0]['events']['times']
voltage = nest.GetStatus(voltmeter)[0]['events']['V_m']
pl.plot(times, voltage)
pl.xlabel('time (ms)'); pl.ylabel('membrane potential (mV)')
pl.show()
```

A FULL EXAMPLE



JÜLICH Forschungszentrum

EVENT-DRIVEN VS. TIME-DRIVEN

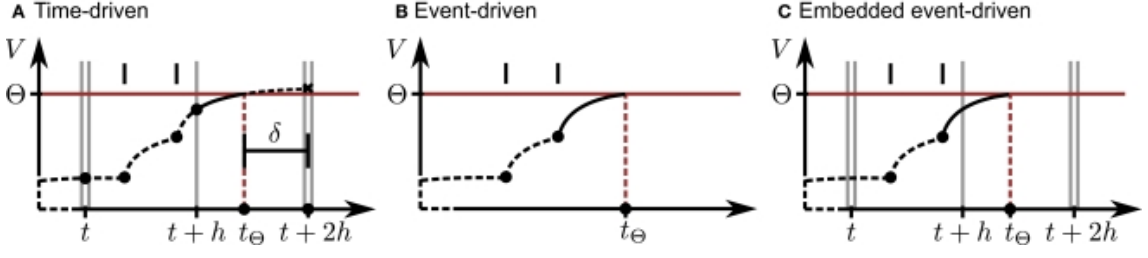
Event-driven simulation:

- Visit a neuron only when it receives an event (e.g. a spike)
- From $y(t_i)$, calculate $y(t_{i+1})$

Time-driven simulation:

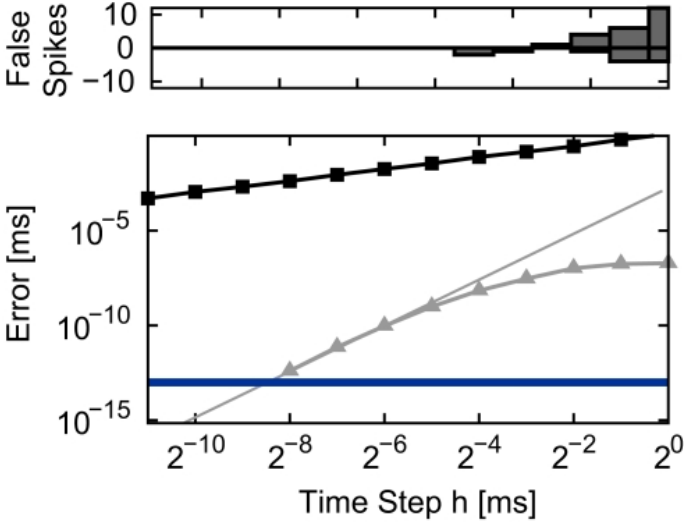
- Visit each neuron in each time step h
- From $y(ih)$, calculate $y([i+1]h)$

EVENT-DRIVEN VS. TIME-DRIVEN



Hanuschkin et al. Front. Neuroinf. 2010

EVENT-DRIVEN VS. TIME-DRIVEN



Hanuschkin et al. Front. Neuroinf. 2010

JÜLICH Forschungszentrum

EVENT-DRIVEN VS. TIME-DRIVEN

	Event-driven	Time-driven
Pros	■ more efficient for low input rates ■ 'correct' solution for invertible neuron models	■ more efficient for high input rates ■ works for all neuron models ■ scales well
Cons	■ only works for neurons with invertible dynamics ■ event queue does not scale well	■ only 'approximate' solution even for analytically solvable models ■ spikes can be missed due to discrete sampling of membrane potential

EVENT-DRIVEN VS. TIME-DRIVEN

NEST uses a hybrid approach to simulation

- input events to neurons are frequent: time-driven algorithm
  - If the dynamics is nonlinear, we need a numerical method to solve it, e.g.:
    - Forward Euler: $y([i+1]h) = y(ih) + h \cdot \dot{y}(ih)$
    - Runge-Kutta (k-th order)
    - Runge-Kutte-Fehlberg with adaptive step size
    - ...

  → Use a pre-implemented solver, for example, from the GNU Scientific Library (GSL).

  - If the dynamics is linear (e.g. leaky integrate-and-fire), we can solve it exactly.

EVENT-DRIVEN VS. TIME-DRIVEN

NEST uses a hybrid approach to simulation

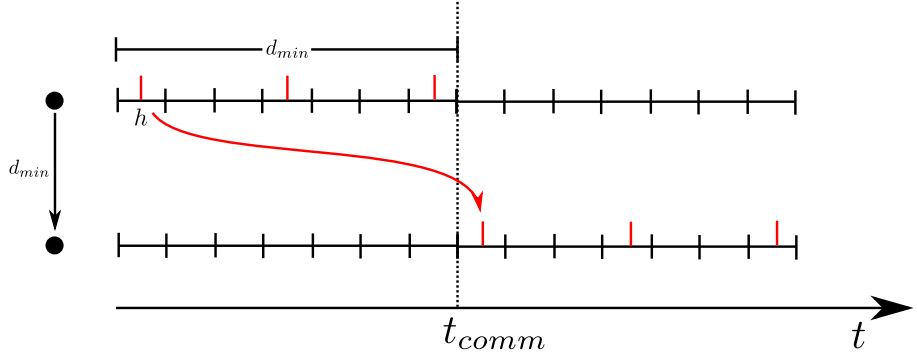
- input events to neurons are frequent: time-driven algorithm
  - If the dynamics is nonlinear, we need a numerical method to solve it, e.g.:
    - Forward Euler: $y([i+1]h) = y(ih) + h \cdot \dot{y}(ih)$
    - Runge-Kutta (k-th order)
    - Runge-Kutte-Fehlberg with adaptive step size
    - ...

  → Use a pre-implemented solver, for example, from the GNU Scientific Library (GSL).

  - If the dynamics is linear (e.g. leaky integrate-and-fire), we can solve it exactly.
- events at synapses are rare: event driven component
  - Exception: gap junctions

MINIMUM SYNAPTIC DELAY

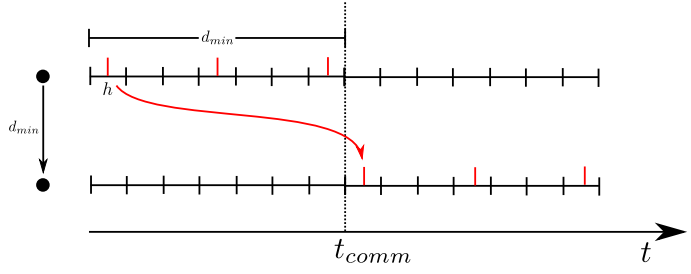
During an interval of the minimal transmission delay in the network (Δ), neurons are effectively decoupled.



Input events to neurons are frequent: time-driven updates.
Events at synapses are rare: event driven.

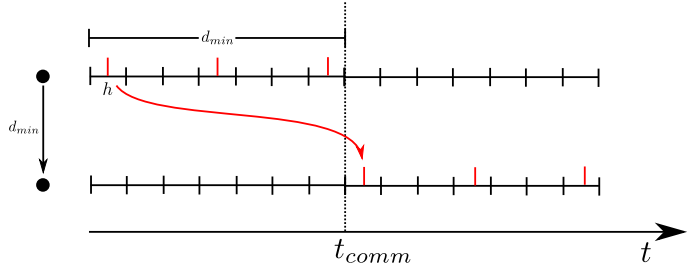
COMMUNICATION OF EVENTS

- communication only required in intervals of the minimal delay between neurons



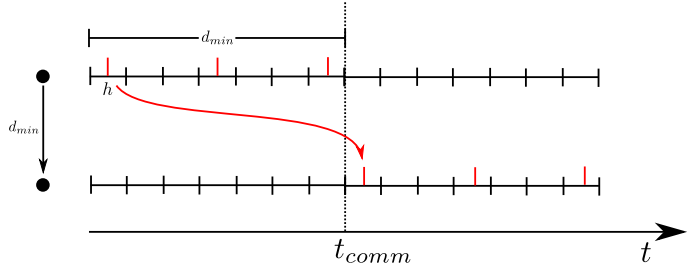
COMMUNICATION OF EVENTS

- communication only required in intervals of the minimal delay between neurons
- communication frequency independent of step size *h*



COMMUNICATION OF EVENTS

- communication only required in intervals of the minimal delay between neurons
- communication frequency independent of step size *h*
- less communications containing more data is more efficient due to overhead of communication between machines



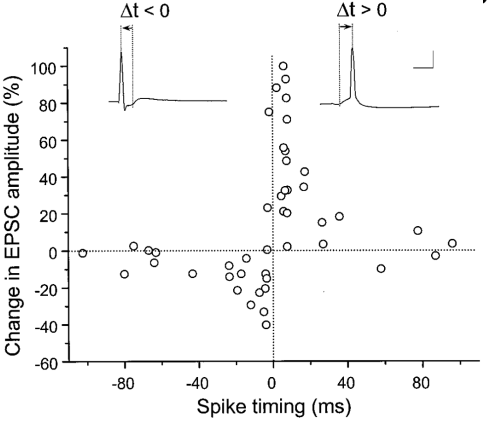
SYNAPTIC PLASTICITY

- Biological substrate of learning and memory
- ”Synaptic update rules” or ”learning rules” describe the change of connection strength w_{ij} for a synapse from neuron j to neuron i
- Different time scales for inducing a change and the persistence of the change, e.g.,
  - Short-term plasticity: induced in less than a second, lasts less than a second
  - Long-term depression (LTD) and long-term potentiation (LTP): induced in a few seconds, can last for more than an hour
- Dependence on, e.g., pre- and postsynaptic spike times, or additionally a third factor as modulatory signal, for example, neuromodulation, eligibility traces, voltage

Morrison, Diesmann & Gerstner (2008)

PLASTICITY MECHANISMS IN NEST

Spike-timing dependent plasticity (STDP)



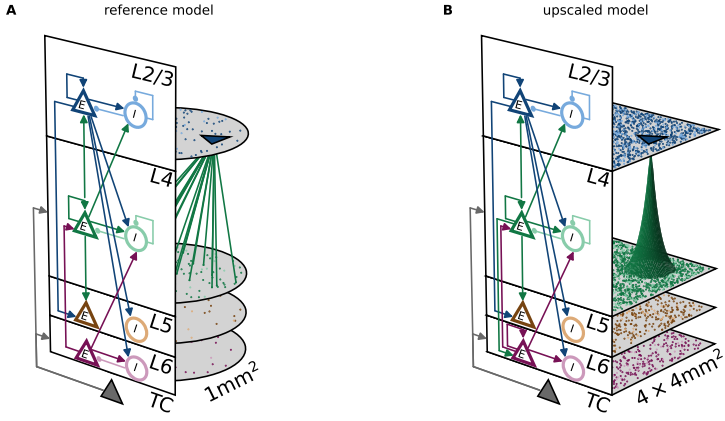
Bi & Poo (1998)

- All-to-all and several nearest-neighbour variants
- Triplet STDP (Pfister & Gerstner, 2006)
- Voltage-based STDP (Clopath et al., 2010)
- Dopamine-modulated STDP (Potjans et al., 2010)

PLASTICITY MECHANISMS IN NEST

- Short term depression and facilitation (Tsodyks, Uziel & Markram, 2000)
- Voltage-based learning rules (Clopath et al., 2020; Urbanczik & Senn, 2014)
- Structural plasticity (Butz & van Ooyen, 2013)
- E-prop (biologically plausible approximation of back-propagation through time) (Bellec et al., 2020)

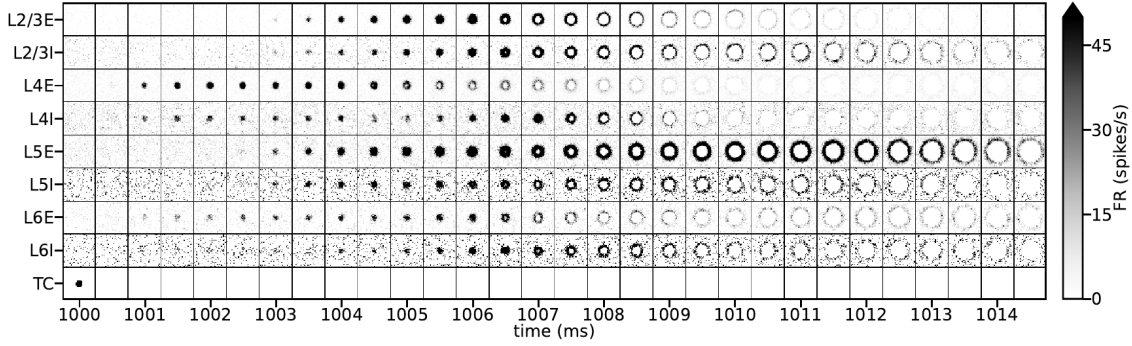
MESOCIRCUIT MODEL



- $4 \times 4\text{mm}^2$ similar to Utah array
- 3.1 million neurons, 7.7 billion synapses
- Distance-dependent connection probabilities (exponential profile) and delays

Senk, Hagen, van Albada & Diesmann (2023) arXiv v2

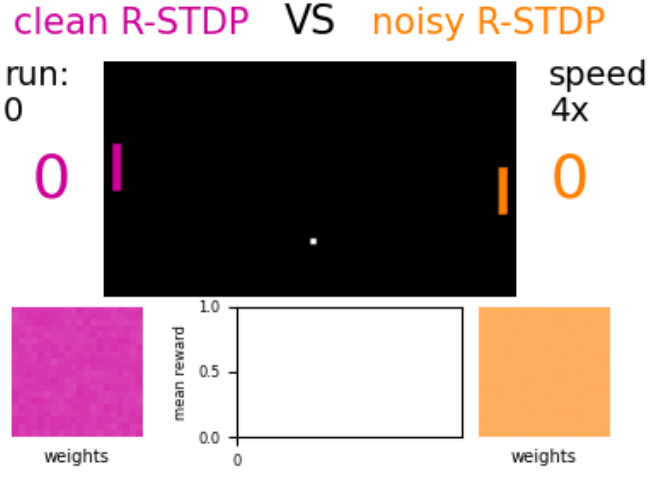
EVOKED ACTIVITY: THALAMO-CORTICAL STIMULATION



Senk, Hagen, van Albada & Diesmann (2023) arXiv v2

LEARNING TO PLAY PONG

Two spiking neur
output mapping i



Wunderlich T., et al (2019). Demonstrating advantages of neuromorphic computation: a pilot study. Frontiers in neuroscience, 13, 260

NEST GPU

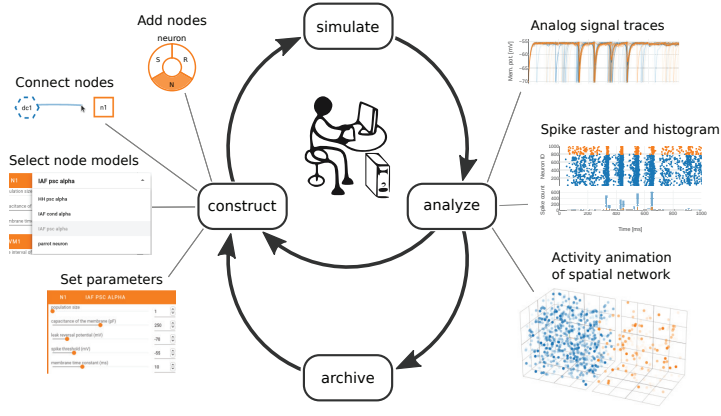
Recently introduced extensions allowing NEST to perform GPU-accelerated simulations.
`https://github.com/nest/nest-gpu`



Golosio, Tiddia, De Luca, Pastorelli, Simula & Paolucci (2021) Front. Comput. Neurosci.
Tiddia, Golosio, Albers, Senk, Simula, Pronold, Fanti, Pastorelli, Paolucci & van Albada (2022) Front. Neuroinform.
Golosio, Villamar, Tiddia, Pastorelli, Stapmanns, Fanti, Paolucci, Morrison & Senk (2023) Appl. Sci.

NEST DESKTOP—A GUI TO NEST

- Web-based
  graphical user interface
  as educational application
- Interactive construction and
  exploration of neural networks



Spreizer, Senk, Rotter, Diesmann & Weyers (2021) eNeuro

`https://nest-desktop.readthedocs.io`
`https://ebrains.eu/service/nest-desktop`

LARGE COMMUNITY WITH OVER 30 YEARS OF EXPERIENCE

SYNOD: An Environment for Neural Systems Simulations

Language Interface and Tutorial

Markus Diesmann[1], Marc-Oliver Gewaltig[2], and Ad Aertsen[1]



- User mailing list
- Bi-weekly developer VC
- Annual virtual conference (16+17 June 2026)
- Papers based on NEST simulations

ACKNOWLEDGMENTS

This presentation is based on previous work by many people:

- Hannah Bos
- David Dahmen
- Moritz Deger
- Jochen Martin Eppler
- Espen Hagen
- Charl Linssen
- Abigail Morrison
- Hans Ekkehard Plesser
- Jannis Schuecker
- Tom Tetzlaff
- Sacha van Albada